Java™

Contents

Ingeniería de Datos para Biosferas Planetarias 3

I. Prólogo 4

II. Introducción 5

V. Ejercicio 0: El Punto de Entrada 6

VI. Ejercicio 1: Tipos de Fallos en la Estación 7

VII. Ejercicio 2: Alarmas Personalizadas 8

VIII. Ejercicio 3: El Bloque de Seguridad Final 10

IX. Ejercicio 4: Protocolo de Autodestrucción 11

X. Ejercicio 5: El Protocolo de Reentrada 12

XIII. Ejecución 13

Ingeniería de Datos para Biosferas Planetarias

En el espacio, un error de datos no solo es un bug, es una catástrofe. Aprenderás a construir tuberías de datos (pipelines) resilientes para gestionar estaciones biológicas en Marte y Europa, manejando fallos de sensores y tormentas solares sin que el sistema colapse.

Instrucciones Generales

- ✓ ***Versión:*** Java 17+ (OpenJDK).
- ✓ ***Estructura:*** Una clase por archivo (Puedes usar package).
- ✓ ***Linter Obligatorio:*** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ ***Comentarios:*** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ ***IDE:*** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ ***Compilación:*** Vía terminal: javac Clase.java y java Clase.
- ✓ **Regla de Supervivencia:** Tus programas ***nunca deben cerrarse inesperadamente (crash)***. Debes capturar los errores y recuperarte.

Entrega y Evaluación

- Entrega en repositorio Git.

I. Prólogo

El Centinela del Vacío

En la inmensidad del espacio, los datos son el oxígeno de la supervivencia. Building en tus bases de monitorización (Módulo 01), ahora te enfrentas a los desafíos reales de la agricultura inteligente en entornos hostiles. En una estación orbital o en una base en Marte, los datos fluyen como el agua a través de sistemas de irrigación: lecturas de sensores constantes, previsiones de radiación solar y telemetría de humedad en invernaderos digitales.

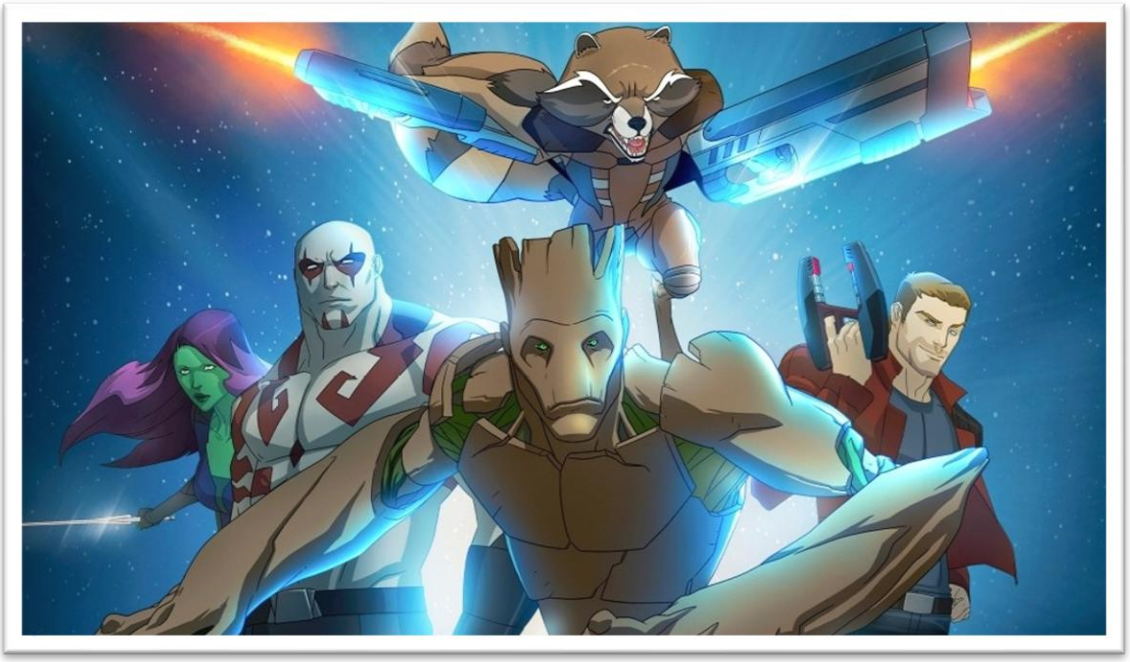
Pero ¿qué sucede cuando los sensores fallan en plena cosecha? ¿Qué ocurre cuando las conexiones se pierden durante una tormenta solar crítica o cuando datos corruptos amenazan con activar ciclos de riego falsos que agotarían nuestras reservas? Los ingenieros de datos espaciales profesionales saben que los sistemas robustos no se construyen para evitar fallos, sino para manejarlos con elegancia cuando ocurre lo inesperado.

Tu invernadero digital debe ser tan resiliente como la propia naturaleza. El sistema de manejo de excepciones de Java es tu caja de herramientas para construir tuberías de datos a prueba de balas. Aprenderás a capturar anomalías, crear alertas personalizadas y asegurar la integridad de la misión incluso cuando las leyes de Murphy ataquen en el vacío. En este proyecto, evolucionarás de un programador básico a un ingeniero de datos capaz de mantener la vida creciendo sin importar lo que pase.

II. Introducción

¡Te damos la bienvenida a **Garden Guardian: ¡Ingeniería de Datos Espaciales!**

Partiendo de los cimientos de monitorización que construiste anteriormente, ahora dominarás las habilidades críticas para la ingeniería de tuberías de datos resilientes en sistemas de soporte vital.



A lo largo de este módulo, descubrirás:

- Cómo validar y limpiar flujos de datos agrícolas en tiempo real.
- Qué tipos de modos de fallo existen en las redes de sensores interestelares.
- Cómo crear alertas personalizadas para cultivos específicos en condiciones de baja gravedad.
- Técnicas esenciales para la tolerancia a fallos y la recuperación de sistemas críticos.
- Cómo garantizar la integridad de los datos en sistemas de cultivo distribuidos por toda la galaxia.

Cada ejercicio construye un componente de tu plataforma de datos de agricultura espacial, progresando desde la validación básica de sensores hasta sistemas de monitorización integrales y automáticos. Tus programas deben demostrar cómo construir sistemas que manejen los escenarios del mundo real con gracia y precisión.

V. Ejercicio 0: El Punto de Entrada

- **Directorio:** ex0/
- **Archivo:** FtOxygenValidator.java
- **Funciones Autorizadas**: try, catch, Double.parseDouble(), System.out.println(), NumberFormatException.
- **Método**: public static void main(String[] args).
- **Restricción:** No puedes usar if (isNumeric) (si existiera). Debes intentar la conversión y capturar el error si falla.

Objetivo: Filtrar lecturas corruptas de los sensores de la base.

- Crea un método *public static void checkOxygenLevel(String input)*
- que intente convertir la lectura a un número.
- **Rango seguro:** 18% <= a <= 25%. Si está fuera, lanza una excepción.

Salida Esperada:

```
=== Oxygen Sensor Telemetry ===
Input '21.5': [OK] Level at 21.5% – Optimal conditions.
Input '15.0': [ALERT] 15.0% is below critical threshold (Min 18%)
Input '99.9': [ALERT] 99.9% is oxygen toxicity risk! (Max 25%)
Input 'abc': [ERROR] Sensor corrupt: 'abc' is not a valid numeric
reading.
```

VI. Ejercicio 1: Tipos de Fallos en la Estación

- **Directorio:** ex1/
- **Archivo:** FtSpaceStationErrors.java
- **Método**: public static void main(String[] args).
- **Funciones Autorizadas:** try, catch, Integer.parseInt(), System.out.println(),.
- **Restricción:** No se permite evitar el error con un if. Debes provocarlo (ej. dividir por cero a propósito) y luego capturarlo con el bloque catch adecuado.

Objetivo: Demostrar que sabes identificar diferentes desastres técnicos.

- Provoca y captura específicamente:
  - ***NumberFormatException***: Datos de telemetría corruptos.
  - ***ArithmeticException***: Intento de calcular trayectoria con división por cero.
  - ***ArrayIndexOutOfBoundsException***: Acceso a un módulo de la estación que no existe.
- **Requisito:** El programa debe seguir funcionando después de cada error.

Salida Esperada:

```
=== Station Diagnostic Run ===
1. Testing Telemetry...
   Caught: NumberFormatException - Data packet corrupted.
2. Testing Navigation...
   Caught: ArithmeticException - Trajectory division by zero.
3. Testing Module Access...
   Caught: ArrayIndexOutOfBoundsException - Module does not exist.

Diagnostic complete. System Status: STABLE (Errors handled).
```

VII. Ejercicio 2: Alarmas Personalizadas

- **Directorio:** ex2/
- **Archivo:** FtPlanetExceptionsTry.java, FtPlanetExceptionsThrows.java, FtPlanetRunExceptions.java
- **Método**: Any.
- **Funciones Autorizadas:** try, catch, System.out.println(), Exception (Checked), RuntimeException (Unchecked), equals, getMessage, .class.getSimpleName()

Objetivo: Crear tus propios protocolos de emergencia.

- ***FtPlanetExceptionsTry.java (Exception)*** – (Opción con try-catch)
- ***FtPlanetExceptionsThrows.java (Exception)*** – (Opción con throws)
- ***FtPlanetRunExceptions (RuntimeException)***

- Crea una jerarquía de excepciones:
  - ***StationException*** (Base)
  - ***SolarStormException*** (Hereda de StationException)
  - ***OxygenLeakException*** (Hereda de StationException)

Deben ir todas las excepciones en un mismo archivo, por ello para este problema ***puedes ignorar el error de Linter:*** Clase de nivel superior XXXXException tiene que residir en su propio archivo de origen.

Todas tus excepciones deben aceptar un mensaje (un String) para explicar qué está pasando.

- **Reto:** Demuestra que sabes gestionar e implementar errores personalizados, para los 3 casos. En el **main**, lanza las excepciones hijas, pero demuestra que puedes atraparlas.

1era salida esperada - FtPlanetExceptionsTry

```
=== Mission Control Alert System ===

Triggering: OxygenLeakException
Report: [CRITICAL] Pressure drop in Sector 7G!
Captured as Station: [CRITICAL] Pressure drop in Sector 7G!

Triggering: SolarStormException
Report: [WARNING] High radiation detected. Deploying shields
Captured as Station: [WARNING] High radiation detected. Deploying
shields
```

2da salida esperada – FtPlanetExceptionsThrows

```
=== Mission Control Alert System ===
Triggering: OxygenLeakException
Report: [CRITICAL] Pressure drop in Sector 7G!
Captured as OxygenLeakException: [CRITICAL] Pressure drop in Sector
7G!

Triggering: SolarStormException
Report: [WARNING] High radiation detected. Deploying shields
Captured as SolarStormException: [WARNING] High radiation detected.
Deploying shields
```

3ra salida esperada – FtPlanetRunExceptions

```
=== Mission Control: Runtime System ===
Triggering: OxygenLeakException
Caught as: OxygenLeakException
Report: [CRITICAL] Pressure drop in Sector 7G!
-----------------------------------
Triggering: SolarStormException
Caught as: SolarStormException
Report: [WARNING] High radiation detected.
-----------------------------------
```

VIII. Ejercicio 3: El Bloque de Seguridad Final

- **Directorio:** ex3/
- **Archivo:** FtShieldSystem.java
- **Funciones Autorizadas:** System.out.println().
- **Restricción:** Es obligatorio que el mensaje de "Cerrando escudos" aparezca dentro del bloque finally, no fuera del try-catch.
-

Objetivo: Asegurar que los sistemas críticos se cierren correctamente, incluso si hay una explosión.

- Simula la apertura de los escudos térmicos.
- Si ocurre un error durante la calibración, el escudo **debe cerrarse siempre** en el bloque finally para proteger a la tripulación.

Salida Esperada:

```
=== Shield Deployment Sequence ===
Attempting to calibrate thermal shields...
[!] ERROR: Calibration failed due to sensor interference.
[MANDATORY] Closing thermal shields for crew safety...
Sequence finished.
```

IX. Ejercicio 4: Protocolo de Autodestrucción

- **Directorio:** ex4/
- **Archivo:** FtEmergencyProtocol.java
- **Método:** Any
- **Funciones Autorizadas:** Palabra clave throw.
- **Restricción:** No imprimas el error con System.out. Debes **lanzar** la excepción para que el programa se detenga si no hay un catch.

Objetivo: Forzar al sistema a detenerse cuando detectas un sabotaje.

- Si el reactorTemperature supera los 1000°C, usa throw new RadiationLeakingException("Critical Overheat!").

Salida Esperada:

```
=== Reactor Core Monitor ===
Current Temp: 800°C – Stable.
Current Temp: 1200°C
Exception in thread "main" RadiationLeakingException:
Critical Overheat! Core at 1200°C.
```

X. Ejercicio 5: El Protocolo de Reentrada

- **Directorio:** ex5/
- **Archivo:** FtReentryProtocol.java
- **Funciones Autorizadas:** Palabra clave throws (en la firma del método) y try-catch (solo en el main).
- **Restricción:** El método calculateHeatShield prohibido tener un bloque catch. Su única misión es detectar el error y lanzarlo hacia arriba.
- **Concepto nuevo:** Delegación de errores con la palabra clave throws.

Escenario:
Estás programando el sistema de cálculo de escudo térmico para la reentrada en la atmósfera. Si los cálculos fallan, el sistema de navegación debe ser notificado inmediatamente para abortar.

Requisitos:

- ✓ Crea un método llamado calculateHeatShield(int angle).
- ✓ Este método **no tendrá un bloque try-catch** interno. En su lugar, usará throws AngleCriticalException en su firma para avisar que puede fallar.
- ✓ Si el ángulo es menor a 10 o mayor a 45 grados, lanza la excepción (throw new ...).
- ✓ En el método main (la computadora central), llama a calculateHeatShield. Aquí **sí** debes usar un try-catch para capturar el error y activar el protocolo de emergencia.

Salida Esperada:

```
=== Deep Space Navigation ===
Testing Reentry Angle: 25... [SUCCESS] Ship is on track.
Testing Reentry Angle: 60...
[EMERGENCY] AngleCriticalException: 60.0° exceeds safety
limits!
[SYSTEM] Aborting reentry. Recalculating orbit...
```

XIII. Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando javac te dirá "***archivo no encontrado***" y te entrarán ganas de tirar el PC por la ventana.

- ✓ cd Desktop/proyectos_java/ex00
- ✓ ***javac*** FtHelloForest.java **-> La Compilación (El traductor)**
- ✓ ***java*** FtHelloForest.java **-> La Ejecución (El lanzamiento)**

Término Java	**Concepto en la Misión**
try	Intentar una operación arriesgada (ej. aterrizar).
catch	El escudo que detiene el impacto del error.
finally	Protocolo de limpieza que se ejecuta pase lo que pase.
throw	Apretar el botón de alarma roja manualmente.
throws	Avisar en el manual del método que esto podría explotar.